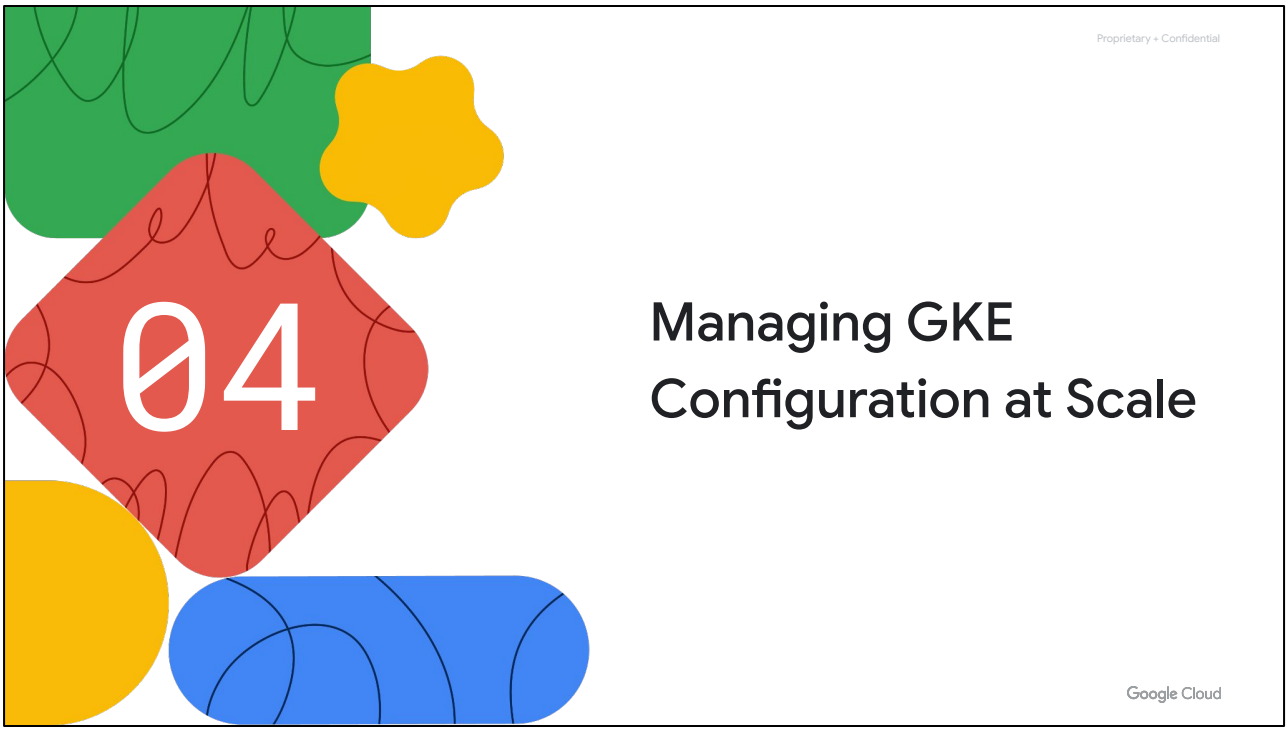


Say: As organizations increasingly adopt Kubernetes and GKE to orchestrate their containerized applications, the complexity of managing configurations across multiple clusters and teams grows exponentially.

Imagine a scenario with dozens of GKE clusters spread across different teams and projects. Ensuring consistency, security, and efficiency in such an environment can quickly become a nightmare without the right tools and strategies. Thankfully, GKE includes a suite of tools and services.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

06 Blueprints

Google Cloud

Say: In this section of the course, titled “Managing GKE configuration at Scale,” you’ll learn to:

- Recognize the challenges of scaling multi-cluster, multi-tenant configurations.
- Configure a centralized configuration management using GitOps model.
- Describe the benefits and architecture of Config Sync.
- Use Policy Controller to enforce security and compliance in GKE.
- Extend GitOps principles to Google Cloud resources.
- Create a standardized, reusable, and policy-driven foundation for Kubernetes deployments.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

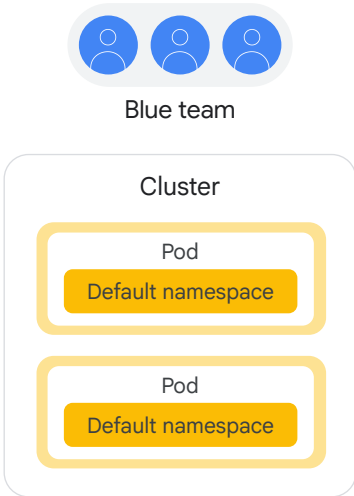
06 Blueprints

Google Cloud

Say: Configuration management in GKE can be challenging, especially at enterprise scale.

Proprietary + Confidential

Simple configurations work for small teams

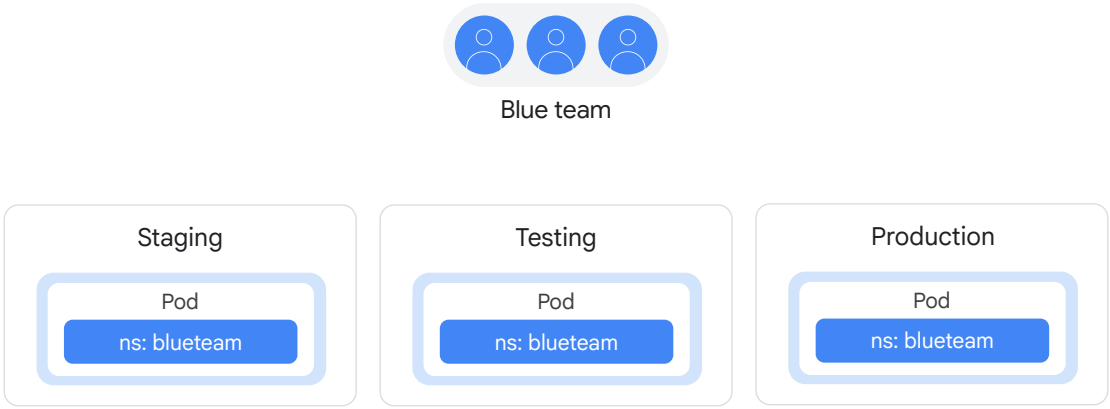


Google Cloud

Say: For small teams, a simple configuration, like a cluster for each dev team, can be sufficient.

Proprietary + Confidential

The need for multiple clusters



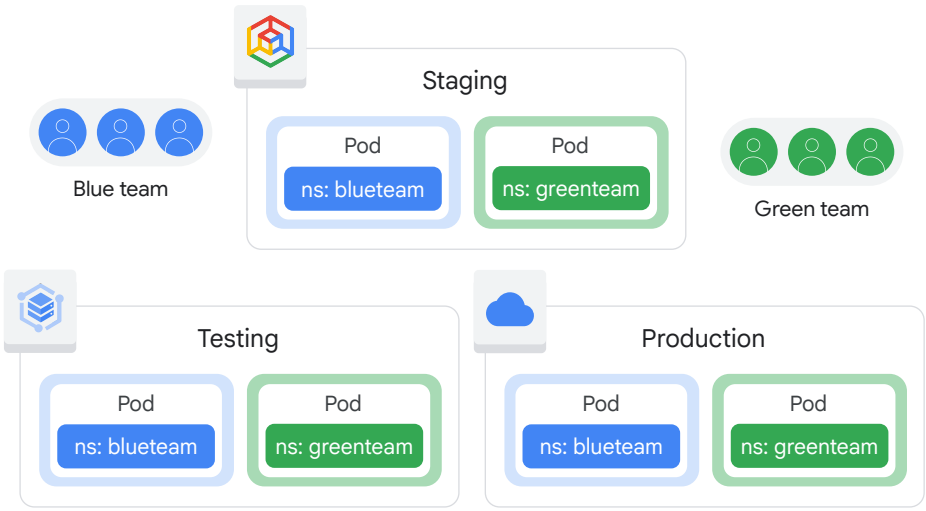
Google Cloud

Say: But as teams outgrow their cluster, you need to provision the same environment across multiple clusters.

For example, your teams might need a staging, testing, and production environment.

Proprietary + Confidential

Multiple tenants and clusters increases complexity



Google Cloud

Say: Adding tenants, namespaces, and clusters increases the amount of policy applications your team needs to manage.

Furthering the complexity is the need for compliance across geographically distributed environments.

Proprietary + Confidential

Drift: desired state and observed state do not match

- Adding or removing resources
- Changing resource definitions
- Resource termination or failure

Google Cloud

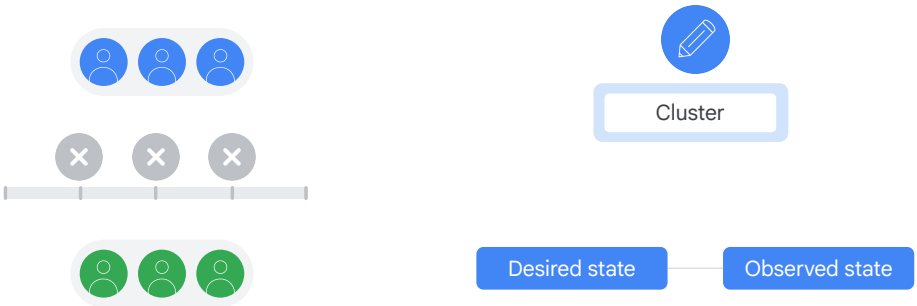
Say: Managing configuration is challenging, and over time, you might experience drift.

Drift is when the real-world state of your infrastructure differs from the state defined in your configuration.

Adding or removing resources, changing resource definitions, resource termination or failure—these are all examples of factors that might cause drift.

Proprietary + Confidential

The need for a centralized repository



Kubernetes manages resources declaratively,
but changes are applied imperatively.

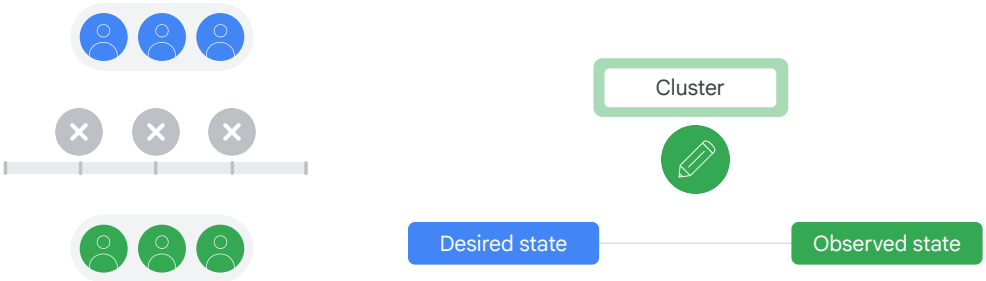
Google Cloud

Say: Kubernetes manages resources declaratively, but changes are applied imperatively.

For example, let's say there are two teams, blue and green. The blue team modifies the state of an individual Kubernetes cluster, but doesn't store the changes in a centralized repository.

Proprietary + Confidential

The need for a centralized repository



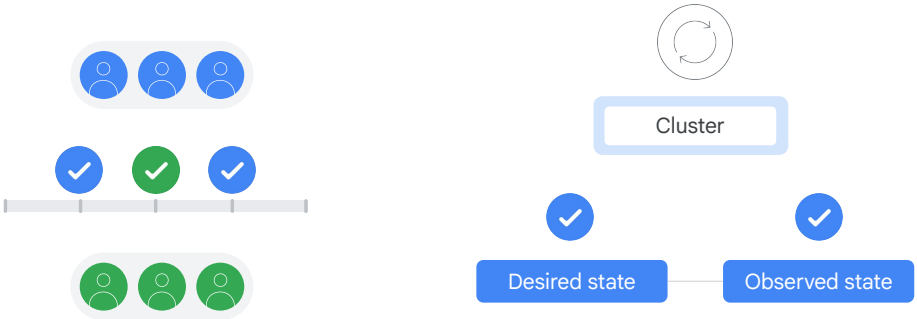
As changes are made, it becomes difficult to determine what changes need to be made to reach the desired state.

Google Cloud

Say: Because the green team is unaware of the changes, their future changes override the changes made by the blue team. As time goes on and more changes are made, the actual state of the cluster and the desired state start to separate. It becomes difficult to determine what changes need to be made to reach the desired state.

Proprietary + Confidential

Manage configuration declaratively with GitOps



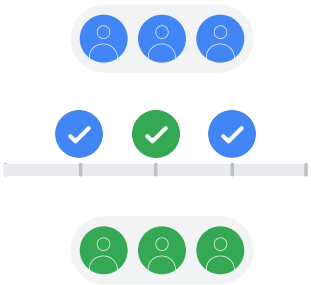
You can manage configuration declaratively with GitOps.

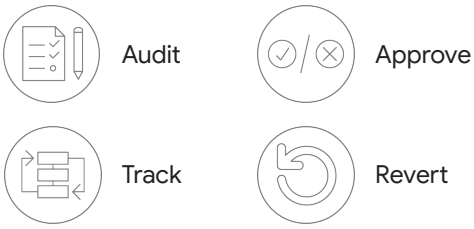
Google Cloud

Say: To solve this challenge, you can manage configuration declaratively with GitOps. Instead of individual teams or tools directly modifying the state of a cluster, changes are made and stored in a shared, centralized repository. Clusters retrieve the configuration continuously.

Proprietary + Confidential

GitOps: centralized configuration





Google Cloud

Say: The centralized configuration can be audited, tracked, approved, and reverted, providing high visibility and control over cluster changes.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

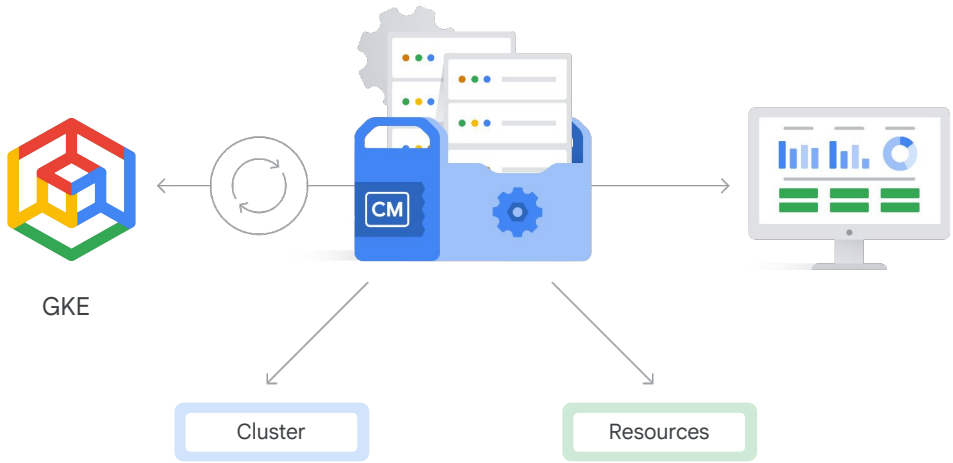
06 Blueprints

Google Cloud

Say: To support the GitOps methodology, Google offers several tools and services.

Proprietary + Confidential

GKE can sync configurations and policies



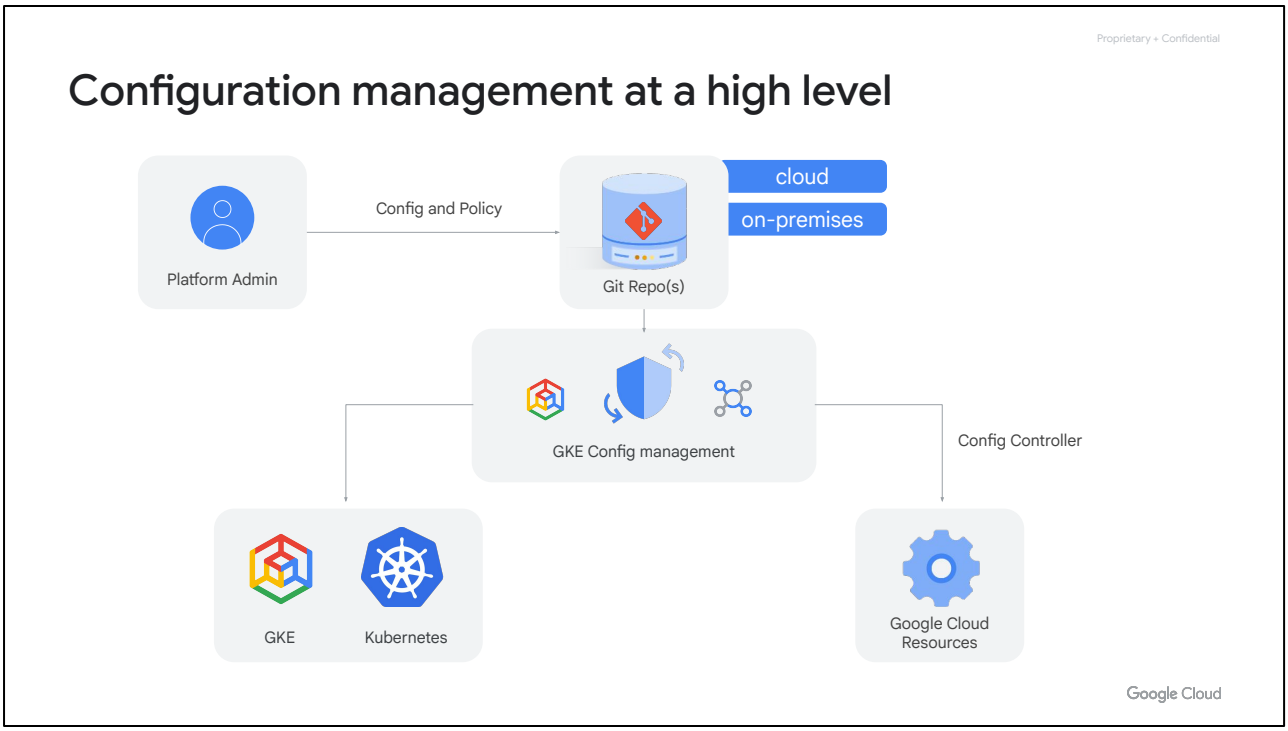
Google Cloud

Say: After platform administrators shift to a central repository, GKE can automatically synchronize configurations and policies across clusters and cloud resources.

Then you can use Config Controller to view details like resource status and issues from a centralized dashboard.

Config Controller is a fully managed service that allows you to manage Google Cloud resources using Kubernetes-style declarative configuration. Think of it as extending Kubernetes' powerful configuration management to your entire Google Cloud infrastructure.

Let explore the benefits that Config Controller bring:



Say: Let's explore the GKE configuration management process at a high level.

First, the platform administrator creates policies and configurations and pushes them to a Git repository. The repository can either be hosted in the cloud or on-premises.

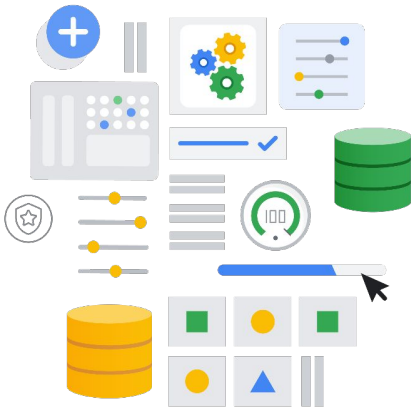
Then GKE Config Sync synchronizes and applies the changes in the GKE clusters.

GKE Policy Controller enforces the applied policies, whether they were custom created or selected from the default policy library.

Then Config Controller creates resources in Google Cloud, extending the configuration beyond just Kubernetes.

Proprietary + Confidential

The benefits of Configuration and Policy Management



- There is no need to build tools from scratch, simplifying management.
- Consistent configuration and policy management.
- Scalable, automated, and reliable management of configurations and systems in production.
- Reduce risk with customizable and consistent policies across environments.

Google Cloud

Say: One benefit is simplified management. Multi-cluster configurations and policies can be managed centrally, without needing to build tools from scratch.

Another benefit is consistent configuration and policy management. You can audit and manage cluster configurations within a version-controlled environment.

Config Controller also provides centralized governance for scalable, automated, and reliable management of configurations and systems in production.

And Config Controller is secure and consistent. Reduce risk with customizable and consistent policies across environments. Monitor environments to ensure configurations are implemented and governance controls are present.

Proprietary + Confidential

Configuration and Policy Management Elements

Config Sync

Policy Controller

Config Controller

Google Cloud

Say: To achieve this high level of configuration management, Google uses Config Sync, Policy Controller, and Config Controller.

Proprietary + Confidential

Config Sync

Config Sync

- Continuously syncs clusters to a central set of of configurations stored in one or more Git repositories.
- Ensures consistent, auditable, and version-controlled configurations across clusters and environments.

Google Cloud

Say: Config Sync continuously syncs your clusters to a central set of configurations stored in one or more Git repositories. This centralized system ensures consistent, auditable, and version-controlled configurations across clusters and environments.

Proprietary + Confidential

Policy Controller

Policy Controller

- Enforces programmable policies that define constraints on the desired state.
- Prevent configurations from violating security and compliance controls.
- Features a library of pre-built policies for common security and compliance controls.
- Recommended practices can be used review and maintain configuration compliance.

Google Cloud

Say: Policy Controller enforces programmable policies that define constraints on the desired state.

These policies act as "guardrails" and prevent configurations from violating security and compliance controls. For example, you can block non-compliant API requests or audit the configuration of your clusters and report violations.

Policy Controller is part of the Open Policy Agent Gatekeeper project and features a library of pre-built policies for common security and compliance controls.

Policy management best practices can be used to review and maintain configuration compliance.

Proprietary + Confidential

Config Controller

Config Controller

- Uses an API endpoint to provision and orchestrate GKE and Google Cloud resources.
- This component can also be run in your cluster.

Google Cloud

Say: Finally, Config Controller uses an API endpoint to provision and orchestrate GKE and Google Cloud resources. This component can also be run in your cluster.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

06 Blueprints

Google Cloud

Say: Config Sync enables cluster operators and platform administrators to deploy consistent configurations and policies directly from a git repo. This concept is called configuration as code.

Proprietary + Confidential

GitOps and Config Sync benefits

Config Sync

- **Auditability:** every change is in Git and correlates with a specific commit.
- **Revertability:** Config Sync keeps a record of changes so you can roll back to a previous state.
- **Transactional:** changes can be grouped and applied at once.
- **Self-Healing:** Config Sync guards from drift and ensures that the declaration in the repository matches the cluster.

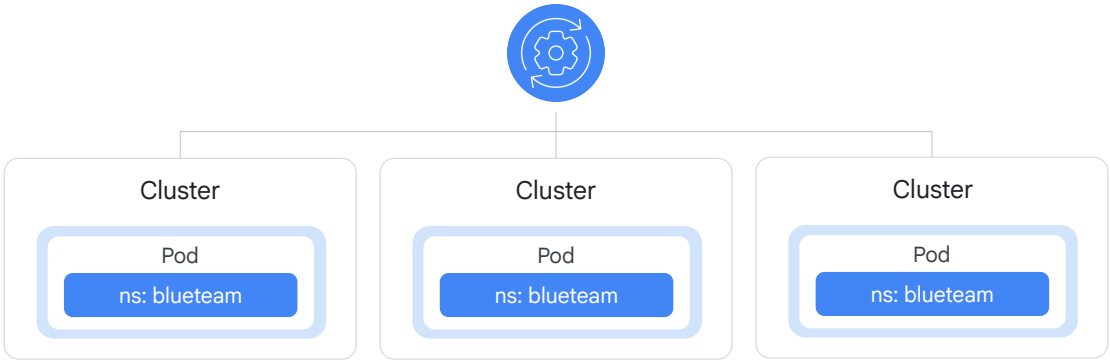
Google Cloud

Say: Let's explore the benefits of Config Sync.

- Auditability: every change is in Git and correlates with a specific commit.
- Revertability: Config Sync keeps a record of changes so you can roll back to a previous state.
- Transactional: changes can be grouped and applied at once.
- Self-Healing: Config Sync guards from drift and ensures that the declaration in the repository matches the cluster.

Proprietary + Confidential

Config Sync ensures consistency

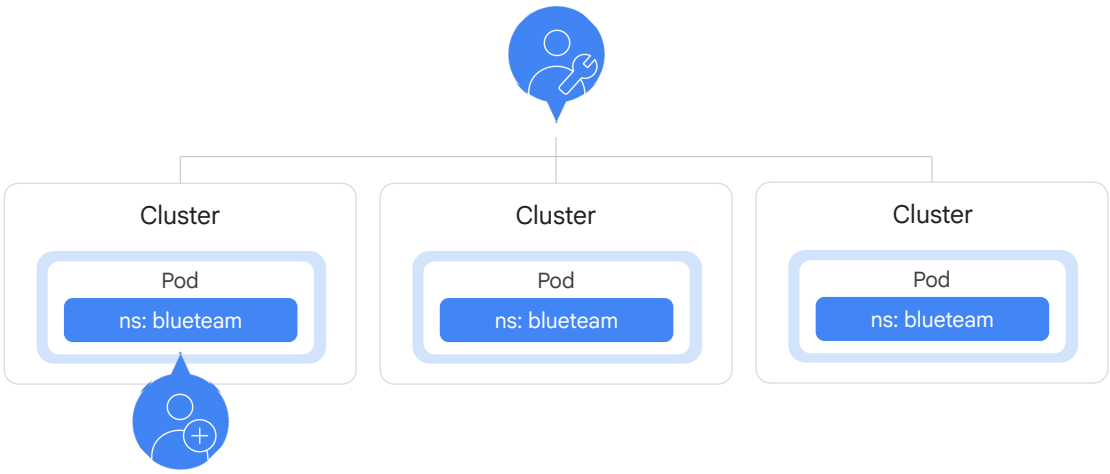


Google Cloud

Say: Config Sync ensures consistent configurations and policies across clusters and namespaces, simplifying multi-tenancy.

Proprietary + Confidential

Config Sync ensures consistency



Google Cloud

Say: A central administrator oversees infrastructure and policies, and application operators manage deployments within their namespaces.

Proprietary + Confidential

Meet Olivia, a platform administrator

Olivia

Platform administrator

- Ensure that all platform infrastructure across all teams is compliant with government requirements.
- Administer project clusters, which are essential infrastructure elements.

Google Cloud

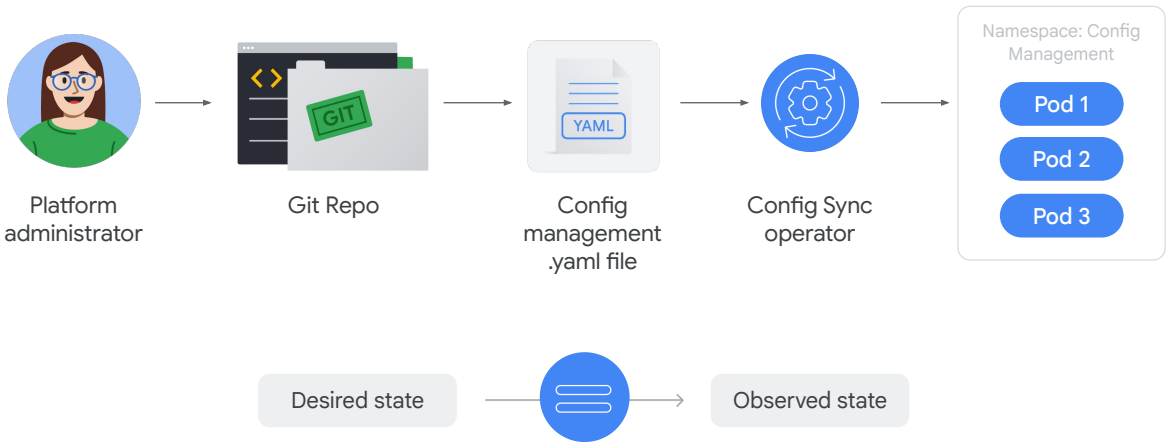
Say: Before we transition and examine the Config Sync architecture, let's explore a use case.

Meet Olivia. Olivia has just been given a new role as platform administrator. Her first task is to ensure that all platform infrastructure across all teams is compliant with government requirements.

Part of Olivia's responsibility is to administer project clusters, which are essential infrastructure elements.

Proprietary + Confidential

Cluster administration process



Google Cloud

Say: Olivia starts by configuring a Git repo to store the Configs, so that all clusters can sync with the central repository.

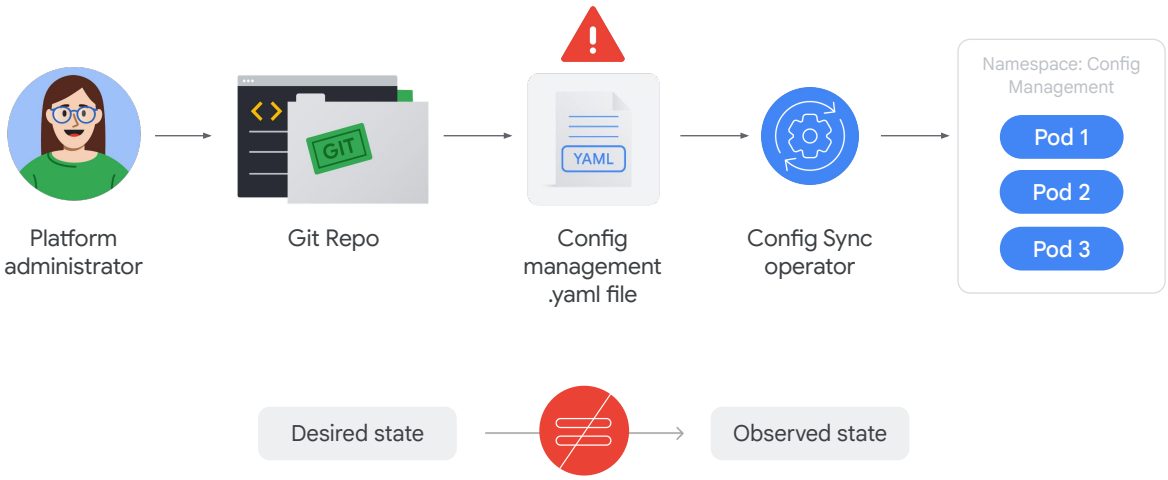
Then she creates a Config Management YAML file that will be applied to all clusters. This file tells GKE which clusters to operate on, and provides information about the git repository, including which brand and folder to sync from.

The Config Sync operator creates Pods in the Config Management namespace. These Pods ensure that the desired state of the cluster matches the actual state.

The infrastructure defined in the repo contains instructions for creating testing and development namespaces.

Proprietary + Confidential

Config Sync recognizes errors



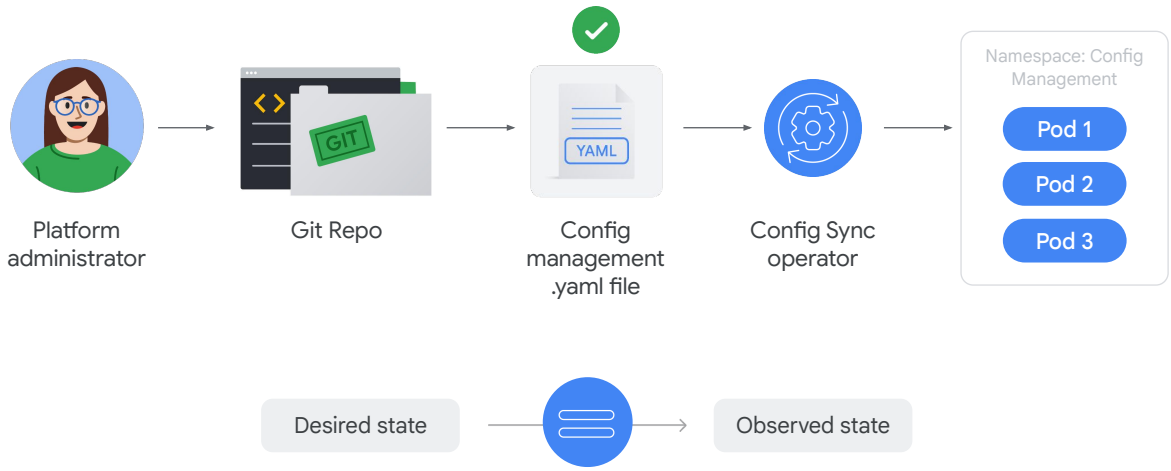
Google Cloud

Say: A member of Olivia's team was inspecting the configuration file and accidentally deleted one of the namespaces.

Luckily, Config Sync recognizes that the actual and desired state are out of sync within a few seconds of the deletion

Proprietary + Confidential

Config Sync ensures desired and observed state match



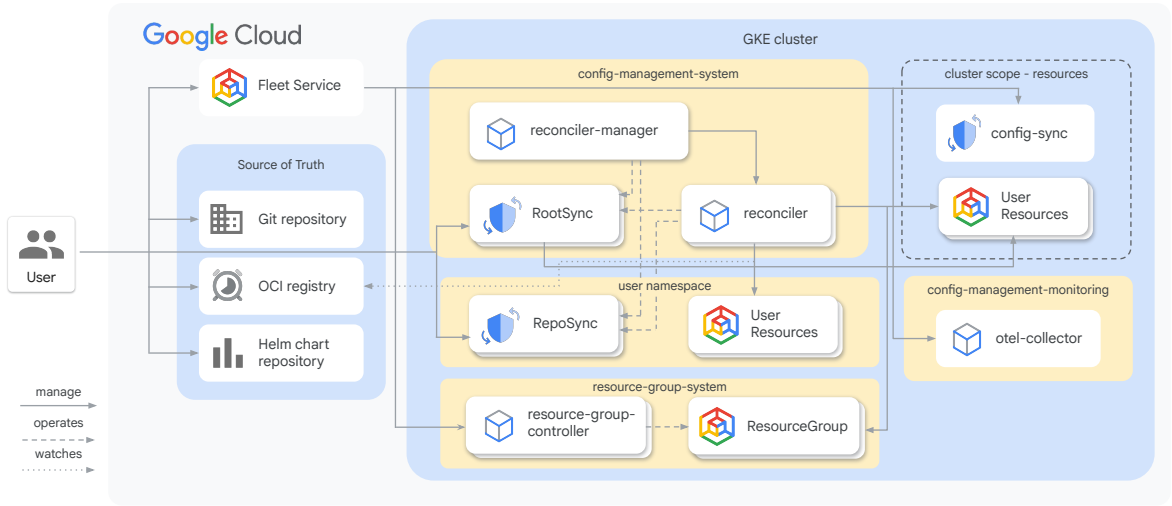
Google Cloud

Say: and creates a new version of the deleted namespace to restore the actual infrastructure with the declarative configuration.

Config Sync combines a GitOps approach with Git storage and versioning. Any changes will be automatically synced to the cluster for consistency.

Proprietary + Confidential

Config Sync architecture



Google Cloud

Say: Next let's explore the Config Sync architecture. Config Sync consists of multiple deployments, each with specific roles and container configurations.

Components like ConfigMangement Operator, Reconciler Manager, and OpenTelemetry Collector are deployed during Config Sync installation.

ConfigManagement Operator and objects: they manage Config Sync components and require cluster-admin permissions.

Reconciler Manager and reconcilers: they create and manage reconcilers for RootSync and RepoSync objects, ensuring synchronization.

Reconciler containers: they include reconciler, otel-agent, and sync containers (git-sync, helm-sync, oci-sync), with an optional hydration-controller and gcenode-askpass-sidecar containers.

ResourceGroup Controller and ResourceGroup objects: they monitor and update the status of synced objects within each RootSync or RepoSync.

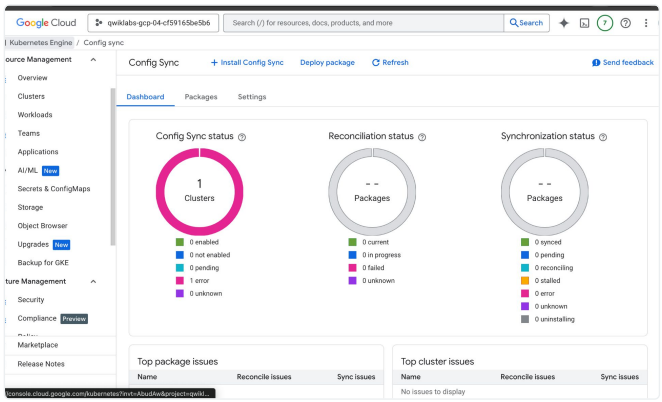
Admission Webhook: it enforces drift prevention by intercepting and validating configuration changes.

RootSync and RepoSync objects: they define the source of truth and scope for synchronization (cluster-wide or namespace-specific).

Fleet service and RootSync objects: they manage the initial RootSync object and allow for additional RootSync or RepoSync objects.

Proprietary + Confidential

Config Sync dashboard



- Sync status
- Error identification
- Visualizations of resource changes across clusters

Google Cloud

Say: The Config Sync dashboard in GKE provides a centralized view of your configuration deployments.

The Config Sync dashboard provides:

- Sync status
- Error identification
- Visualizations of resource changes across clusters

This provides insight into drift detection and policy enforcement and simplifies multi-cluster configuration management, ensuring consistency and compliance.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

06 Blueprints

Google Cloud

Say: As your Kubernetes deployments grow, ensuring security and compliance across your organization becomes critical. That's where Policy Controller comes in. It acts like a vigilant guard, automatically enforcing policies across your GKE clusters. This helps you prevent misconfigurations, enforce recommended practices, and meet regulatory requirements.

Proprietary + Confidential

Policy Controller

Policy Controller

- Apply native or custom security and operational policies directly in the Kubernetes API.
- All clients are subject to your corporate security and compliance controls.
- Shifts security and compliance enforcement "left"—directly into your Kubernetes infrastructure.
- You define policies within the Kubernetes API itself.

Google Cloud

Say: You can use Policy Controller to apply native or custom security and operational policies directly in the Kubernetes API. That way, all clients are subject to your corporate security and compliance controls.

Policy Controller essentially shifts security and compliance enforcement "left"—directly into your Kubernetes infrastructure. Instead of relying on external tools or manual checks, you define policies within the Kubernetes API itself. This means, no matter how users or applications interact with your GKE cluster (kubectl, Terraform, CI/CD pipelines, and so forth), they must adhere to these rules.

Proprietary + Confidential

Shift left benefits

Consistent enforcement

Policies become an integral part of your cluster configuration.

Early detection

Violations are caught immediately during deployment or configuration changes.

Centralized control

Define policies once and apply them across your cluster.

Improved posture

Prevent misconfigurations and ensure compliance with industry standards or internal regulations.

Google Cloud

Say: The benefits of this “shift left” approach are:

- Consistent enforcement: policies become an integral part of your cluster configuration. No more inconsistencies or workarounds.
- Early detection: violations are caught immediately during deployment or configuration changes, preventing issues before they reach production.
- Centralized control: define policies once and apply them cluster-wide, simplifying management and reducing the risk of human error.
- Improved security posture: prevent misconfigurations and ensure compliance with industry standards or internal regulations.

Proprietary + Confidential

Default policy library

- Define custom policies or choose from a library of built-in default policies.
- Library is continually expanded and maintained by the Policy Controller team.

Only allow domain-restricted Roles and RoleBindings.

Require strict mTLS for all clients/services in a namespace.

Enforce specific labels for cost accounting and chargeback.

Require fine-grained service authorization controls.

Require services to disable unauthorized access.

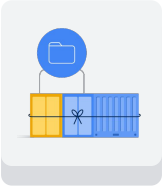
Require access logging to be enabled for a cluster/mesh.

Google Cloud

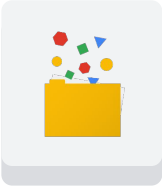
Say: Policy controller gives you the option to define custom policies or choose from a library of built-in default policies. The library is maintained by Google, and new policies are frequently added to meet the growing compliance requirements of enterprise organizations.

Proprietary + Confidential

Default policy library



Restrict Pod security



Enforce resource limits

Control network access

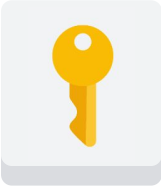
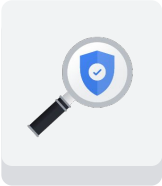


Image security



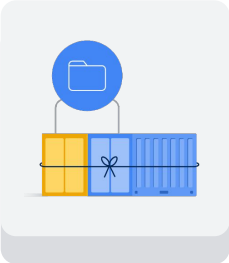
Compliance and auditing

Google Cloud

Say: Let's explore some of the ways in which Policy Controller can help ensure security and compliance.

Proprietary + Confidential

Restrict Pod security



Built in policy

Require all pods to run with a non-root user, preventing privilege escalation attacks.

Custom Policy

Disallow containers from running with certain capabilities to limit their network access.

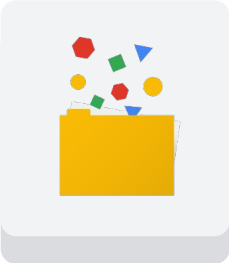
Google Cloud

Say: Restrict Pod security:

- Built-in policy: require all pods to run with a non-root user, preventing privilege escalation attacks.
- Custom policy: disallow containers from running with certain capabilities (e.g., NET_ADMIN) to limit their network access.

Proprietary + Confidential

Enforce resource limits



Built in policy

Ensure all deployments specify resource requests and limits (CPU, memory) to prevent resource exhaustion.

Custom Policy

Define maximum resource limits for specific namespaces or applications to optimize resource allocation.

Google Cloud

Say: Enforce resource limits:

- Built-in policy: ensure all deployments specify resource requests and limits (CPU, memory) to prevent resource exhaustion.
- Custom policy: define maximum resource limits for specific namespaces or applications to optimize resource allocation.

Proprietary + Confidential

Control network access

Built in policy

Enforce network policies to restrict pod-to-pod communication.

Custom Policy

Require all services to have a specific label for network traffic management.

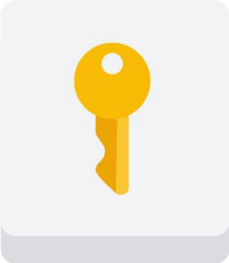
Google Cloud

Say: Control network access:

- Built-in policy: enforce network policies to restrict pod-to-pod communication, segmenting your applications for better security.
- Custom policy: require all services to have a specific label (for example, environment: production) for network traffic management.

Proprietary + Confidential

Image security



Built in policy

Require image pull policies to be set to always to ensure you're using the latest image versions.

Custom Policy

Only allow images from trusted registries or with specific security scan results.

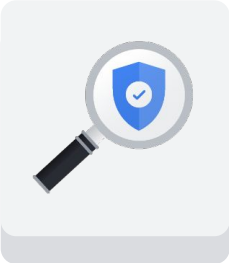
Google Cloud

Say: Image security:

- Built-in policy: require image pull policies to be set to always to ensure you're using the latest image versions.
- Custom policy: only allow images from trusted registries or with specific security scan results.

Proprietary + Confidential

Compliance and auditing



Built in policy

Enforce compliance with PCI DSS, HIPAA, or other regulatory requirements by using policies aligned with those standards.

Custom Policy

Create audit trails by requiring specific annotations on resources, providing evidence for compliance audits.

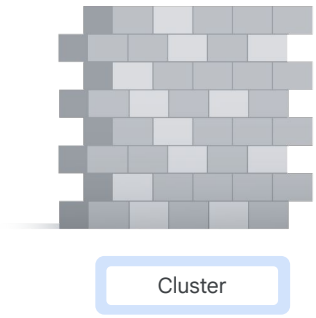
Google Cloud

Say: Compliance and auditing:

- Built-in policies: enforce compliance with PCI DSS, HIPAA, or other regulatory requirements by using policies aligned with those standards.
- Custom policies: create audit trails by requiring specific annotations on resources, providing evidence for compliance audits.

Proprietary + Confidential

Policy constraints

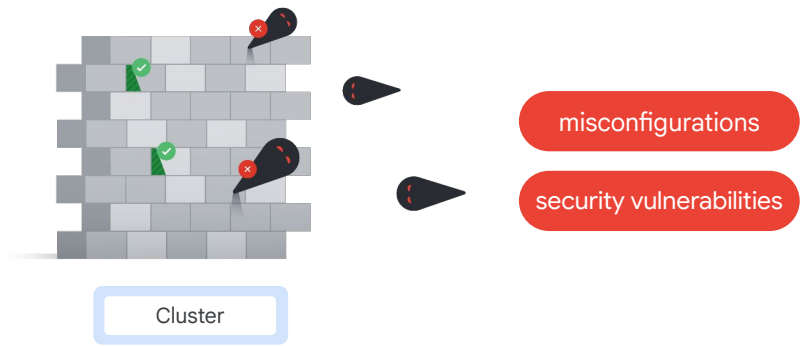


Google Cloud

Say: Next, let's explore constraints. In Kubernetes, policy constraints help you enforce rules and standards across your cluster.

Proprietary + Confidential

Policy constraints

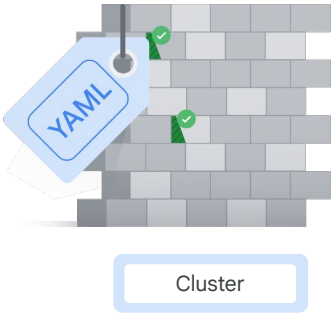


Google Cloud

Say: Think of them as guardrails that prevent misconfigurations or security vulnerabilities.

Proprietary + Confidential

Policy constraints



Constraints are defined using YAML.

Google Cloud

Say: You define a constraint using YAML, so you don't need to know Rego, the underlying policy language, to create these constraints.

Proprietary + Confidential

Policy constraints

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: ns-must-have-team
spec:
  match:
    kinds:
    - apiGroups: [“”]
      kinds: [“Namespace”]
  parameters:
    labels:
    - key: [“team”]
```

The **constraint template** provides the logic for the constraint, while you provide specific parameters to tailor it to your needs.

If you organize your Kubernetes configurations using a structured repository, it's recommended to store your constraints in the **cluster/** directory.

Google Cloud

Say: A constraint uses a pre-defined "constraint template." This template provides the logic for the constraint, while you provide specific parameters to tailor it to your needs. This makes constraints reusable and easier to manage.

If you organize your Kubernetes configurations using a structured repository, it's recommended to store your constraints in the cluster/ directory.

Proprietary + Confidential

Policy constraints

```
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sRequiredLabels
metadata:
  name: ns-must-have-team
spec:
  match:
    kinds:
    - apiGroups: [""]
      kinds: ["Namespace"]
  parameters:
    labels:
    - key: ["team"]
```

A policy constraint typically identifies:

- The kind, which is the type of constraint template, in this case K8sRequiredLabels.
- The name of the constraint, in this case ns-must-have-team.
- A match field that determines which resources the constraint applies to. Match conditions include kinds, namespaces, exclude namespaces, labelselector, and namespaceselector.
- Parameters that contain specific values or settings for customization.

Google Cloud

Say: A policy constraint typically identifies:

- The kind, which is the type of constraint template, in this case K8sRequiredLabels
- The name of the constraint, in this case ns-must-have-team
- A match field that determines which resources the constraint applies to. Match conditions include kinds, namespaces, exclude namespaces, labelselector, and namespaceselector
- Parameters that contain specific values or settings for customization.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Controller

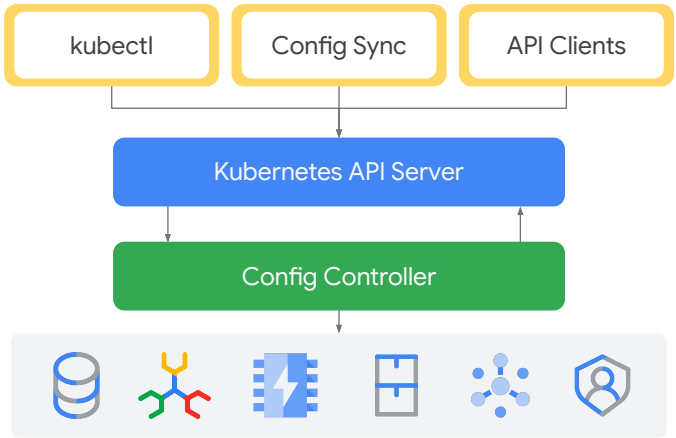
06 Blueprints

Google Cloud

Say: Next let's explore Config Controller. Managing cloud resources alongside your Kubernetes deployments can be a challenge. Config Controller simplifies this by bringing your Google Cloud resources directly into the Kubernetes ecosystem.

Proprietary + Confidential

Config Controller



Google Cloud

Say: Think of it as a universal translator for Kubernetes and Google Cloud. Config Controller wraps your Google Cloud resources like Cloud SQL databases, Pub/Sub topics, and IAM roles in a declarative Kubernetes API, making them available to all your clients. Manage your Google Cloud services using the same Kubernetes YAML files and tools you use for your applications.

This not only streamlines your workflow but also enables GitOps recommended practices for your entire infrastructure, including your cloud resources.

Proprietary + Confidential

Config Controller

Manage cloud infrastructure with Kubernetes tooling.

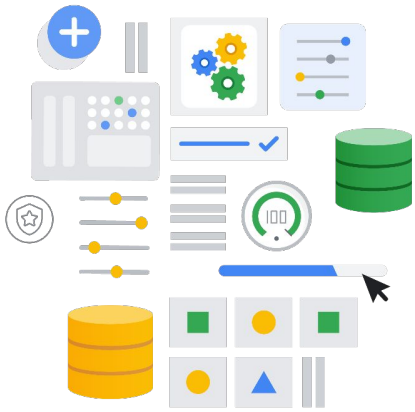
- Kubernetes-native API surface for Google Cloud.
- Specify intended state with declarative configuration.
- Provision cloud infrastructure with Kubernetes APIs or kubectl.
- Eventually consistent, idempotent, combats drift.

Say: Treat Google Cloud services like K8s services, deploying your cloud-backed containers against a single API surface.

Benefit from the same GitOps mechanism we used with Config Sync to manage and combat drift in Kubernetes in a broad coverage of Google Cloud APIs.

Proprietary + Confidential

Bring it all together



- Store configuration as code and benefit from collaborative, version-controlled deployments.
- Define custom policies to enforce security recommended practices, compliance requirements, and organizational standards.
- Bring the benefits of GitOps and policy-driven control to your Google Cloud resources, unifying your configuration.

Google Cloud

Say: Store your configurations as code, and benefit from collaborative, version-controlled deployments.

Define custom policies to enforce security recommended practices, compliance requirements, and organizational standards, so that developers can focus on building applications, not configuring policy controls.

Bring the benefits of GitOps and policy-driven control to your Google Cloud resources, unifying your configuration.

All of this is available in a single package today: GKE Config Management.

We will look at GKE Config Management later in this course.

Proprietary + Confidential

Managing GKE Configuration at Scale

01 Configuration management challenges

02 Centralized configuration management at scale

03 Config Sync

04 Policy Controller

05 Config Connector

06 Blueprints

Google Cloud

Say: The GKE Config Management suite and all its tools can feel overwhelming. To simplify matters and make things easier, you can use Blueprints.

Proprietary + Confidential

Blueprints

Blueprint

A package of deployable, reusable configurations and policies that provide a starting point for your deployments.

- Quickly set up secure and compliant clusters with the right policies and configurations in place.
- Reuse code for fast deployment of declarative systems and environments.
- Shareable and version-controlled.

Google Cloud

Say: Like architectural blueprints that lay out the plans for development, blueprints in Google Cloud provide a foundation for Kubernetes deployments.

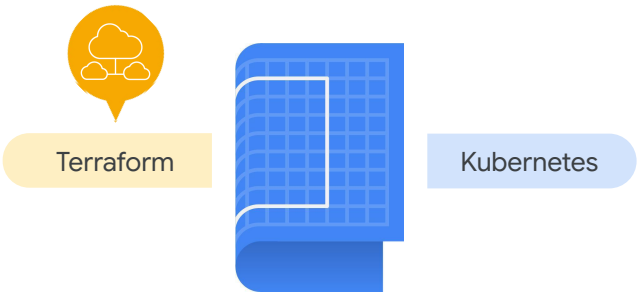
A blueprint is a package of deployable, reusable configurations and policies that provide a starting point for your deployments, helping you quickly set up secure and compliant clusters with the right policies and configurations in place.

With blueprints, you can reuse code for fast deployment of declarative systems and environments.

Blueprints are shareable and version-controlled, and they can be updated without disrupting production.

Proprietary + Confidential

Google provides pre-built blueprints



You can use Terraform modules to extend your management to other cloud providers.

Google Cloud

Say: To get started quickly and easily with GKE, Google provides pre-built blueprints for both Kubernetes and Terraform. These blueprints help you set up and manage your Kubernetes clusters and Google Cloud resources, and you can even use Terraform modules to extend your management to other cloud providers.

Proprietary + Confidential

A landing zone provides a pre-configured foundation

- **Organization:** how your accounts and resources are structured.
- **Permissions:** Who has access to what.
- **Billing:** how costs are managed and tracked.
- **Logging:** how activity is recorded for auditing and troubleshooting.
- **Networking:** how resources connect to each other and the outside world.

Google Cloud

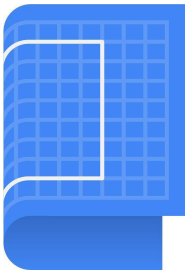
Say: A landing zone provides a pre-configured foundation for your cloud environment, covering crucial aspects like:

- Organization: how your accounts and resources are structured.
- Permissions: who has access to what.
- Billing: how costs are managed and tracked.
- Logging: how activity is recorded for auditing and troubleshooting.
- Networking: how resources connect to each other and the outside world.

Instead of starting from scratch, a landing zone offers best-practice blueprints and sensible defaults, speeding up setup and minimizing the need for manual adjustments. This accelerates onboarding and helps control costs by optimizing resource usage from the get-go.

Proprietary + Confidential

The enterprise application blueprint



Blueprint

Deploy and manage applications within the established landing zone.

- **Define** GKE cluster deployments and configurations.
- **Specify** application architecture (microservices, containers).
- **Establish** CI/CD pipelines.
- **Implement** application-specific security controls.
- **Setup** application monitoring and logging metrics.
- **Define** developer workflows and tooling.

Google Cloud

Say: The enterprise application blueprint focuses on deploying and managing applications within the established landing zone.

It's concerned with the specific architecture, design, and operational aspects of your applications, particularly those running on GKE.

The blueprint helps with:

- Defining GKE cluster deployments and configurations.
- Specifying application architecture (microservices, containers).
- Establishing CI/CD pipelines.
- Implementing application-specific security controls.
- Setting up application monitoring and logging metrics.
- Defining developer workflows and tooling.

The blueprint provides a standardized and repeatable approach to deploying and managing enterprise applications on GKE, enabling developers to efficiently and securely build and deploy applications. It often builds upon the foundation provided by the landing zone.

The landing zone blueprint specifies the environment (where) and the enterprise application blueprint indicates the applications (“what” and “how”).

Proprietary + Confidential

Blueprint benefits

Standardized configuration

Administrators, app operators, and developers all use the same model.

Reusability

Notifies apps when changes are made to a Cloud Storage bucket.

Simplified policy enforcement

Built-in policies are automatically applied and validated early in the process.

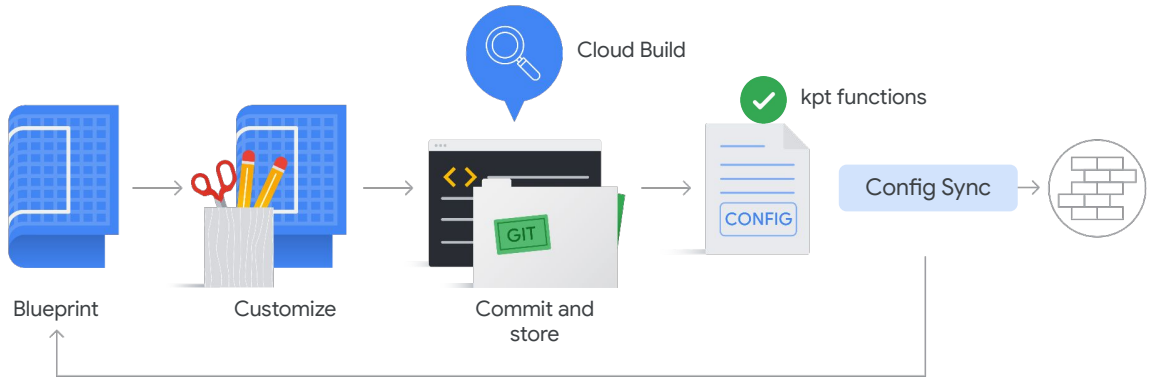
Google Cloud

Say: Key benefits of blueprints include:

- Standardized configuration: administrators, app operators, and developers all use the same model, ensuring everyone is on the same page.
- Reusability: no need to build everything from scratch—simply deploy the blueprint.
- Simplified policy enforcement: blueprints can include built-in policies that are automatically applied and validated early in your development process (CI/CD pipeline).

Proprietary + Confidential

Use blueprints to configure your environment



Google Cloud

Say: Before we wrap up, let's examine how an administrator can use blueprints to set up their environment.

The first step is to choose a blueprint. The administrator explores Google's open-source repository and selects a blueprint that fits their needs. Then they use a tool called kpt to customize the blueprint with their organization's details and ensure the blueprint is tailored to their environment.

The customized blueprint is stored in a version control system (like Git). A Cloud Build trigger is set up to automatically monitor this repository for any changes. Whenever changes are made, automated checks (using kpt functions) ensure the configuration is valid and follows recommended practices.

The final, customized configuration is automatically deployed to their Kubernetes cluster using Config Sync. Config Sync continuously monitors the cluster and ensures it matches the desired configuration defined in the blueprint. Any drift is automatically corrected, keeping the environment in sync.